

MERN COURSE · REFERENCE

The CSS Handbook

The grammar of CSS, its selectors, properties, and units — structured, defined, and shown with examples and diagrams.

AUTHOR & INSTRUCTOR

Saddat Hasan

Full-Stack Engineer · InfinitiBit GmbH

MERN Stack Course · Batch 18 · Companion to Week 2

```
selector { property: value; }
```

Contents

1	Fundamentals	how CSS works, cascade, box model
2	Selectors	who a rule targets
3	Values & Units	how a value is written
4	Typography & Text	fonts, size, spacing
5	Colour & Backgrounds	fills, images, gradients
6	Box: Spacing, Sizing, Borders	margin, padding, radius, shadow
7	Layout: Display & Position	flow, position, z-index
8	Flexbox	one-dimensional layout
9	Grid	two-dimensional layout
10	Transforms, Transitions & Animation	motion & effects
11	At-rules & Responsive	@media, @keyframes, container queries
12	Modern CSS	:has(), nesting, layers, subgrid
13	Quick Reference	the whole grammar on one spread

1 Fundamentals

What CSS is and the rules that govern every stylesheet.

CSS (Cascading Style Sheets) is the language that describes how HTML looks. HTML gives a page its **structure and meaning**; CSS gives it **presentation** — colour, spacing, layout, and motion. The two are kept in separate files on purpose.

■ The shape of every rule

ONE RULE

```
.topic-card {           /* selector - WHO */
  color: #1f2933;        /* property: value */
  padding: 1rem;         /* declaration */
}                        /* declaration block */
```

- **Selector** — which elements the rule targets
- **Property** — what you are changing
- **Value** — what you set it to
- **Declaration** — one property: value; pair, ended by a semicolon
- **Declaration block** — all declarations inside { }

■ Where CSS lives

Three ways to attach CSS

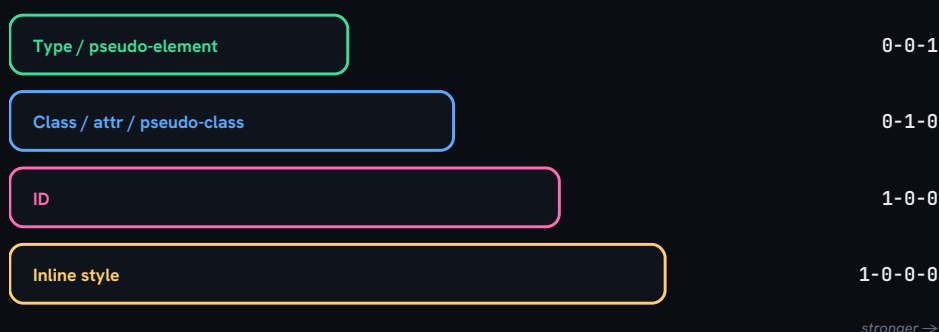
METHOD	HOW	WHEN TO USE
<code>style="..."</code> inline	attribute on one element	quick tests, email; avoid
<code><style></code> internal	a block in the <code><head></code>	a single page
<code><link></code> external	a separate .css file	real projects — one file, every page

THE RECOMMENDED WAY

```
<link rel="stylesheet" href="styles.css">
```

■ The cascade & specificity

When several rules target the same element, two things decide the winner: **specificity** (how targeted the selector is) and, on a tie, **source order** (the later rule wins).



Specificity, weakest to strongest. Higher wins; ties break by order.

Avoid **!important** — it overrides the cascade and makes later bugs hard to trace. Reach for a more specific selector instead.

■ Inheritance

Text-related properties (font, colour, line-height...) set on a parent are **inherited** by its children. Set them once on body and the whole page picks them up. Non-inherited properties (like border) must be set where you want them.

■ The box model

Every element is a box: **content**, wrapped by **padding**, then **border**, then **margin**.



content → padding → border → margin, from inside out.

THE ESSENTIAL RESET

```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

By default `width` covers only the content, so padding and border make elements bigger than expected. **border-box** makes `width` mean the **whole** box — far more predictable.

2 Selectors

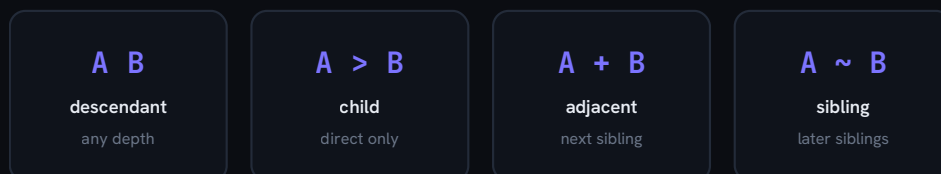
Every way to say *which* elements a rule applies to.

■ Basic selectors

Target by type, class, id, or attribute

SELECTOR	MATCHES	EXAMPLE
<code>*</code>	every element	<code>* { box-sizing: border-box; }</code>
<code>p</code>	by element (type)	<code>p { margin: 1rem 0; }</code>
<code>.card</code>	by class — reusable	<code>.card { border: 1px solid; }</code>
<code>#nav</code>	by id — one unique element	<code>#nav { position: sticky; }</code>
<code>[disabled]</code>	has the attribute	<code>[disabled] { opacity: .5; }</code>
<code>[type="text"]</code>	attribute equals value	<code>input[type="text"] { }</code>
<code>[href^="https"]</code>	value starts with	<code>a[href^="https"] { }</code>
<code>[src\$=".png"]</code>	value ends with	<code>img[src\$=".png"] { }</code>
<code>[class*="col"]</code>	value contains	<code>[class*="col"] { }</code>
<code>h1, h2, h3</code>	grouping — same rules	<code>h1, h2, h3 { font-weight: 700; }</code>

■ Combinators



The four relationships between two selectors.

Select by relationship

COMBINATOR	MEANING	EXAMPLE
<code>A B</code>	B anywhere inside A	<code>nav a { }</code>
<code>A > B</code>	B is a direct child of A	<code>ul > li { }</code>
<code>A + B</code>	B immediately after A	<code>h2 + p { }</code>
<code>A ~ B</code>	any B after A (same parent)	<code>h2 ~ p { }</code>

■ Pseudo-classes — state & position

A colon selects a state or position

PSEUDO-CLASS	MATCHES	EXAMPLE
<code>:hover</code>	pointer is over it	<code>a:hover { }</code>

PSEUDO-CLASS	MATCHES	EXAMPLE
<code>:focus</code> / <code>:focus-visible</code>	focused (keyboard-only)	<code>input:focus-visible { }</code>
<code>:active</code>	being pressed	<code>button:active { }</code>
<code>:visited</code>	a visited link	<code>a:visited { }</code>
<code>:first-child</code> / <code>:last-child</code>	first / last among siblings	<code>li:first-child { }</code>
<code>:nth-child(even)</code>	by position / formula	<code>tr:nth-child(even) { }</code>
<code>:not(.x)</code>	everything except	<code>li:not(.done) { }</code>
<code>:checked</code> / <code>:disabled</code>	form control states	<code>input:checked { }</code>
<code>:required</code> / <code>:valid</code> / <code>:invalid</code>	form validation states	<code>input:invalid { }</code>
<code>:root</code>	the document root (<html>)	<code>:root { --gap: 1rem; }</code>

■ Pseudo-elements — parts of an element

A double colon styles a generated part

PSEUDO-ELEMENT	STYLES	EXAMPLE
<code>::before</code> / <code>::after</code>	generated content	<code>.tag::before { content: "#"; }</code>
<code>::placeholder</code>	input placeholder text	<code>input::placeholder { }</code>
<code>::selection</code>	highlighted text	<code>::selection { background: gold; }</code>
<code>::marker</code>	list bullet / number	<code>li::marker { color: red; }</code>
<code>::first-line</code> / <code>::first-letter</code>	typographic parts	<code>p::first-letter { }</code>
<code>::backdrop</code>	layer behind a dialog	<code>dialog::backdrop { }</code>

■ Relational & list selectors (modern)

Match against lists or children

SELECTOR	MEANING	EXAMPLE
<code>:is(h1, h2, h3)</code>	any in the list (short-hand)	<code>:is(h1,h2) a { }</code>
<code>:where(...)</code>	like <code>:is()</code> but zero specificity	<code>:where(.a,.b) { }</code>
<code>:has(> img)</code>	parent that contains ...	<code>.card:has(img) { }</code>

`:has()` is the long-awaited “parent selector” — style an element based on what it contains or which child is in a given state. Widely supported since 2023.

3 Values & Units

Every value has a type — a length, a colour, or a function.

■ Length units

Absolute vs relative

UNIT	RELATIVE TO	NOTE
px	nothing (absolute)	the everyday pixel
%	the parent	percentage of the parent
em	the element's font-size	compounds when nested
rem	the root font-size	predictable – preferred for type
ch	width of "0"	good for line length (70ch)
vw / vh	1% of viewport w / h	full-screen sizing
vmin / vmax	smaller / larger viewport side	orientation-safe
dvh / svh / lvh	dynamic / small / large vh	mobile address-bar aware
fr	a fraction of free grid space	grid tracks only

■ Colour

Ways to write a colour

FORM	EXAMPLE	NOTE
keyword	red, tomato, transparent	147 named colours
#hex	#4f46e5, #fff, #0008	+ alpha with 8 digits
rgb() / rgba()	rgb(79 70 229 / .5)	red green blue + alpha
hsl() / hsla()	hsl(243 76% 59%)	hue, saturation, lightness
oklch()	oklch(60% .2 275)	perceptually even – modern
color-mix()	color-mix(in oklch, red, blue)	blend two colours
currentColor	border: 1px solid currentColor	reuse the text colour

■ Functions

Values that compute

FUNCTION	DOES	EXAMPLE
var()	read a custom property	color: var(--accent);
calc()	do maths across units	width: calc(100% - 2rem);
clamp()	min, preferred, max	font-size: clamp(1rem, 2vw, 1.5rem);
min() / max()	pick smaller / larger	width: min(100%, 40rem);

FUNCTION	DOES	EXAMPLE
<code>url()</code>	reference a file	<code>background: url(bg.png);</code>
<code>linear-gradient()</code>	a gradient image	<code>linear-gradient(90deg,#a,#b)</code>

■ Custom properties (variables)

DEFINE ONCE, USE EVERYWHERE

```
:root {
  --accent: #4f46e5;
  --space: 1rem;
}
button {
  background: var(--accent);
  padding: var(--space);
}
```

Names starting with `--` are your own reusable values, defined (usually on `:root`) and read with `var()`. Change one line, the whole site updates.

■ Global keyword values

Reset a property

KEYWORD	EFFECT
<code>inherit</code>	take the parent's value
<code>initial</code>	the property's default
<code>unset</code>	inherit if it normally does, else initial
<code>revert</code>	roll back to the browser's stylesheet

4 Typography & Text

Fonts, sizing, spacing, and text decoration.

Core text properties

PROPERTY	SETS	EXAMPLE VALUES
<code>color</code>	text colour	<code>#1f2933</code> · <code>currentColor</code>
<code>font-family</code>	typeface stack	<code>system-ui, sans-serif</code>
<code>font-size</code>	text size	<code>1rem</code> · <code>clamp(1rem, 2vw, 1.5rem)</code>
<code>font-weight</code>	thickness	<code>400</code> · <code>600</code> · <code>700</code> · <code>bold</code>
<code>font-style</code>	normal / italic	<code>italic</code>
<code>line-height</code>	leading (line spacing)	<code>1.6</code>
<code>letter-spacing</code>	tracking	<code>.02em</code> · <code>-.01em</code>
<code>text-align</code>	horizontal alignment	<code>left</code> · <code>center</code> · <code>justify</code>
<code>text-decoration</code>	underline / line-through	<code>underline</code> · <code>none</code>
<code>text-transform</code>	letter case	<code>uppercase</code> · <code>capitalize</code>
<code>text-shadow</code>	shadow behind text	<code>1px 1px 2px #0008</code>
<code>white-space</code>	wrapping behaviour	<code>nowrap</code> · <code>pre</code>
<code>overflow-wrap</code>	break long words	<code>break-word</code>
<code>text-overflow</code>	truncation	<code>ellipsis</code>
<code>font</code>	shorthand for the lot	<code>600 1rem/1.5 system-ui</code>

A SENSIBLE TEXT BASELINE

```
body {
  font-family: system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #1f2933;
}
```

5 Colour & Backgrounds

Fills, images, and gradients behind your content.

Background properties

PROPERTY	SETS	EXAMPLE VALUES
<code>background-color</code>	a solid fill	<code>#fff</code> · <code>transparent</code>
<code>background-image</code>	image or gradient	<code>url(x.png)</code> · <code>linear-gradient(...)</code>
<code>background-size</code>	scaling	<code>cover</code> · <code>contain</code> · <code>100%</code>
<code>background-position</code>	placement	<code>center</code> · <code>top left</code>
<code>background-repeat</code>	tiling	<code>no-repeat</code> · <code>repeat-x</code>
<code>background-attachment</code>	scroll or fixed	<code>fixed</code>
<code>background</code>	shorthand for all	<code>#fff url(x) center/cover no-repeat</code>

GRADIENTS & THE SHORTHAND

```
.hero {
  background:
    linear-gradient(120deg, #4f46e5, #3ddc97);
}

.card {
  background: #fff center/cover
             no-repeat url(photo.jpg);
}
```

A gradient is an **image**, so it lives in `background-image` (or the shorthand).
background-size: cover fills the box, cropping as needed; **contain** fits the whole image in.

6 Box: Spacing, Sizing & Borders

The properties that size and frame every element.

■ Spacing & sizing

Size and space

PROPERTY	SETS	EXAMPLE VALUES
<code>margin</code>	space outside the border	<code>1rem · 0 auto · 8px 16px</code>
<code>padding</code>	space inside the border	<code>1rem · 8px 16px</code>
<code>width / height</code>	box size	<code>320px · 100% · auto</code>
<code>min-width / max-width</code>	lower / upper bound	<code>max-width: 70ch</code>
<code>min-height / max-height</code>	lower / upper bound	<code>min-height: 100vh</code>
<code>box-sizing</code>	what width includes	<code>border-box</code>
<code>aspect-ratio</code>	lock width : height	<code>16 / 9</code>
<code>overflow</code>	clip or scroll excess	<code>hidden · auto · scroll</code>
<code>gap</code>	gutter (flex & grid)	<code>1rem</code>

Shorthand order for margin/padding is **clockwise**: top right bottom left. Two values = vertical horizontal. `margin-inline: auto` centres a fixed-width block.

■ Borders, radius & shadow

Frame an element

PROPERTY	SETS	EXAMPLE VALUES
<code>border</code>	width style colour	<code>1px solid #e5e7eb</code>
<code>border-width / -style / -color</code>	individual parts	<code>2px · dashed · red</code>
<code>border-radius</code>	rounded corners	<code>10px · 999px · 50%</code>
<code>outline</code>	ring that ignores layout	<code>2px solid blue</code>
<code>outline-offset</code>	gap before the ring	<code>2px</code>
<code>box-shadow</code>	drop / inner shadow	<code>0 4px 12px #0002 · inset ...</code>

7 Layout: Display & Position

How boxes flow, stack, and position themselves.

■ Display

The display property

VALUE	BEHAVIOUR
<code>block</code>	full-width, stacks vertically (div, p)
<code>inline</code>	flows in text; ignores width/height (span, a)
<code>inline-block</code>	flows inline but accepts width/height
<code>none</code>	removed from the page entirely
<code>flex</code>	one-dimensional flex layout (see ch. 8)
<code>grid</code>	two-dimensional grid layout (see ch. 9)

■ Position



The five positioning schemes.

The position property

VALUE	BEHAVIOUR
<code>static</code>	default — normal document flow
<code>relative</code>	offset from its normal spot; still occupies its space
<code>absolute</code>	removed from flow; placed vs nearest positioned ancestor
<code>fixed</code>	placed vs the viewport; stays on scroll
<code>sticky</code>	flows normally, then pins when it hits an offset

THE STICKY NAV

```
.week-nav {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

`top` / `right` / `bottom` / `left` place a positioned box; **z-index** controls which overlapping box sits on top (higher = front).

8 Flexbox

One-dimensional layout — a row or a column.



Items lay out along the main axis; align on the cross axis.

■ On the container

Set these on the flex parent

PROPERTY	SETS	EXAMPLE VALUES
<code>display: flex</code>	turn on flexbox	<code>flex</code> · <code>inline-flex</code>
<code>flex-direction</code>	main axis	<code>row</code> · <code>column</code> · <code>row-reverse</code>
<code>flex-wrap</code>	wrap to new lines	<code>wrap</code> · <code>nowrap</code>
<code>justify-content</code>	align along main axis	<code>center</code> · <code>space-between</code>
<code>align-items</code>	align along cross axis	<code>center</code> · <code>stretch</code> · <code>flex-start</code>
<code>align-content</code>	space wrapped lines	<code>space-around</code>
<code>gap</code>	space between items	<code>1rem</code>

■ On the items

Set these on the children

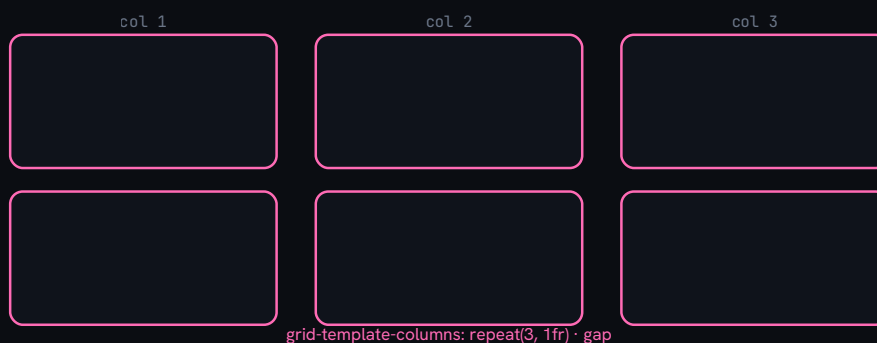
PROPERTY	SETS	EXAMPLE VALUES
<code>flex</code>	grow shrink basis	<code>1</code> · <code>0</code> <code>0</code> <code>auto</code>
<code>flex-grow</code> / <code>-shrink</code> / <code>-basis</code>	the parts	<code>1</code> · <code>0</code> · <code>200px</code>
<code>align-self</code>	override align for one item	<code>flex-end</code>
<code>order</code>	reorder visually	<code>2</code> · <code>-1</code>

FROM STYLES.CSS

```
.week-nav ul {
  display: flex;
  flex-wrap: wrap;
  gap: .5rem;
}
```

9 Grid

Two-dimensional layout — rows and columns together.



Grid defines column and row tracks; items sit in the cells.

■ On the container

Set these on the grid parent

PROPERTY	SETS	EXAMPLE VALUES
<code>display: grid</code>	turn on grid	<code>grid · inline-grid</code>
<code>grid-template-columns</code>	column tracks	<code>1fr 2fr · repeat(3, 1fr)</code>
<code>grid-template-rows</code>	row tracks	<code>100px auto</code>
<code>grid-template-areas</code>	named regions	<code>"head head" "nav main"</code>
<code>gap / row-gap / column-gap</code>	gutters	<code>1rem</code>
<code>justify-items / align-items</code>	align inside cells	<code>center · stretch</code>
<code>grid-auto-flow</code>	auto placement direction	<code>row · column · dense</code>

■ On the items

Set these on the children

PROPERTY	SETS	EXAMPLE VALUES
<code>grid-column / grid-row</code>	span or place	<code>1 / -1 · span 2</code>
<code>grid-area</code>	sit in a named area	<code>main</code>
<code>justify-self / align-self</code>	align one item	<code>start · center</code>

THE SELF-REFLOWING CARD GRID

```
section {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(260px, 1fr));
  gap: 1rem;
}
```

auto-fill + minmax means “fit as many columns as you can, each $\geq 260\text{px}$.” The grid reflows on its own — no media query needed for the columns.

10 Transforms, Transitions & Animation

Move, reshape, and animate elements.

■ Transforms & effects

Reshape and add depth

PROPERTY	DOES	EXAMPLE VALUES
<code>transform</code>	move / rotate / scale / skew	<code>translateY(-3px) · rotate(5deg) · scale(1.05)</code>
<code>transform-origin</code>	the pivot point	<code>center · top left</code>
<code>opacity</code>	transparency	<code>0 - 1</code>
<code>filter</code>	visual filters	<code>blur(4px) · grayscale(1)</code>
<code>backdrop-filter</code>	blur what's behind	<code>blur(8px)</code>
<code>box-shadow</code>	drop shadow	<code>0 6px 20px #0003</code>
<code>object-fit</code>	how media fills its box	<code>cover · contain</code>
<code>cursor</code>	pointer style	<code>pointer · help</code>

■ Transitions

THE CARD LIFT

```
.topic-card {
  transition: transform .15s ease,
             box-shadow .15s ease;
}
.topic-card:hover {
  transform: translateY(-3px);
}
```

A **transition** eases a property from its old value to its new one instead of snapping. Syntax: property duration timing delay.

■ Keyframe animations

DEFINE, THEN RUN

```
@keyframes fade-in {
  from { opacity: 0; }
  to   { opacity: 1; }
}
.toast {
  animation: fade-in .4s ease both;
}
```

@keyframes defines the steps; **animation** runs them. Unlike a transition, an animation can loop and define many steps (0%-100%).

11 At-rules & Responsive

Statements that start with @ — and building for every screen.

Common at-rules

AT-RULE	PURPOSE	EXAMPLE
@import	pull in another stylesheet	@import "reset.css";
@media	rules for a screen condition	@media (max-width: 600px) { }
@font-face	load a custom font	@font-face { font-family: ... }
@keyframes	define an animation	@keyframes spin { ... }
@supports	feature-detect before using	@supports (display: grid) { }
@layer	order the cascade in layers	@layer base, components;
@container	respond to a parent's size	@container (width > 30rem) { }

Media queries — the responsive workhorse

MOBILE-FIRST

```
/* mobile-first: base styles, then enhance */
.grid { grid-template-columns: 1fr; }

@media (min-width: 600px) {
  .grid { grid-template-columns: 1fr 1fr; }
}
```

Write base styles for small screens, then add **@media (min-width)** blocks that enhance for larger ones. Query on width, orientation, prefers-reduced-motion, prefers-color-scheme, and more.

Fluid sizing with `clamp()` can replace many breakpoints entirely: `font-size: clamp(1rem, 2vw + 1rem, 2rem)` scales smoothly with the viewport.

12 Modern CSS

Baseline features worth adopting now.

Recent additions, widely supported

FEATURE	WHAT IT GIVES YOU	EXAMPLE
<code>:has()</code>	style a parent by its children	<code>.card:has(img) { }</code>
<code>:is()</code> / <code>:where()</code>	shorter selector lists	<code>:is(h1,h2,h3) { }</code>
nesting	write child rules inside parents	<code>.card { .title { ... } }</code>
<code>@container</code>	respond to container, not viewport	<code>@container (width>30rem){...}</code>
subgrid	children align to a parent grid	<code>grid-template-columns: subgrid</code>
<code>@layer</code>	tame specificity with cascade layers	<code>@layer base, components;</code>
logical properties	writing-direction-aware box	<code>margin-inline · padding-block</code>
<code>oklch()</code> / <code>color-mix()</code>	better colour maths	<code>color-mix(in oklch, ...)</code>
<code>aspect-ratio</code>	lock proportions, no hacks	<code>aspect-ratio: 16 / 9</code>
<code>accent-color</code>	theme native form controls	<code>accent-color: #4f46e5</code>
<code>dvh</code> / <code>svh</code>	mobile-safe viewport height	<code>height: 100dvh</code>

THE NEW BASELINE

```
/* native nesting + :has() */
.card {
  padding: 1rem;
  &:has(img) { grid-template-columns: 1fr 2fr; }
  .title { font-weight: 700; }
}
```

Nesting removes the need for a preprocessor for most projects; `&` refers to the parent selector. Pair it with `:has()` and container queries for truly component-driven CSS.

13 Quick Reference

The whole grammar, at a glance.

■ The rule, one more time

MEMORISE THIS SHAPE

```
selector {                /* WHO — element, .class, #id, :state */
  property: value;        /* WHAT : HOW — ends with ; */
}
```

■ Selector symbols

SYMBOL	MEANS
.	class
#	id
*	everything
A B / A>B	descendant / direct child
A+B / A~B	adjacent / general sibling
:	state or structure (pseudo-class)
::	a generated part (pseudo-element)
[]	attribute

■ Layout decision

NEED	REACH FOR
a row or column of items, spaced	Flexbox (display: flex)
rows and columns together	Grid (display: grid)
pin something on scroll	position: sticky
overlap / layer boxes	position + z-index
adapt to screen size	@media / clamp()
adapt to container size	@container

Golden rules: keep **structure in HTML**, **style in CSS**; prefer **classes** over ids and **!important**; use **variables** for anything repeated; and let **flex & grid** do layout instead of floats and margins.